pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Case Study: Statement Sentiment

WEEK 4 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note

Analyzing Text Part 2 gave us clean tokens for all 46 FOMC statements. Give me one number per statement for its wording, three ways: two lists we type, a published list, and a list built from the statements' own words. Show me the words behind every number, and whether the three agree. Last: does a change in a score line up with that day's 2 Yr and 10 Yr moves? Describe only: no reading of policy, no view on rates.

A lexicon is a word list; a lexicon score counts its words in a text. Report it as "the list scores the statement of a date at N": a count, never a meaning or a forecast.

	Source	Retrieved
The statements	Board of Governors of the Federal Reserve System, each statement's press release: https://www.federalreserve.gov/monetarypolicy/fomccalendars.htm	2026-10-03
The yields	U.S. Department of the Treasury, Daily Treasury Par Yield Curve Rates:	2026-10-03
	https://home.treasury.gov/resource-center/data-chart-center/interest-rates/TextView?type=daily_treasury_yield_curve	

The statements: 46, 2021-01-27 to 2026-09-16; white space tidied only. The yields: 2015-01-02 to 2026-10-02, in percent.

Every number is printed beside the words that made it.

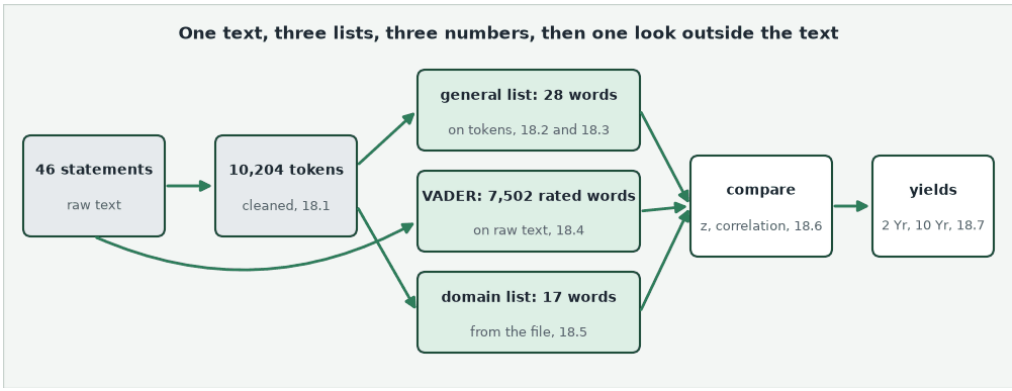
18.1 The statements, cleaned and ready to score

```
statements = pd.read_csv(data_folder + 'fomc_statements.csv')
statements['meeting'] = pd.to_datetime(statements['date'], format='%Y-%m-%d')
print(statements.shape, statements.isna().sum().sum(), statements.duplicated().sum())
(46, 4) 0 0
```

The 46 rows have no gaps and no repeats.

```
def clean(text):
    text = re.sub(r'[^a-z\s]', ' ', text.lower())
    return re.sub(r'\s+', ' ', text).strip().split(' ')
nltk.download('stopwords', quiet=True)
from nltk.corpus import stopwords
stop_words = stopwords.words('english')
def remove_stop_words(tokens):
    return [token for token in tokens if token not in stop_words]
statements['kept'] = statements['text'].apply(clean).apply(remove_stop_words)
statements['n_kept'] = statements['kept'].str.len()
print(statements['n_kept'].sum(), statements['n_kept'].median(), statements['n_kept'].max())
print(statements.nsmallest(4, 'n_kept')[['date', 'n_kept']])
10204 214.0 372
          date  n_kept
45  2026-09-16      79
43  2026-06-17      80
44  2026-07-29     101
40  2026-01-28     178
```

10,204 tokens are kept, Analyzing Text Part 2's count.



The three shortest, 79 to 101 tokens, are the last three statements. Keep that in mind.

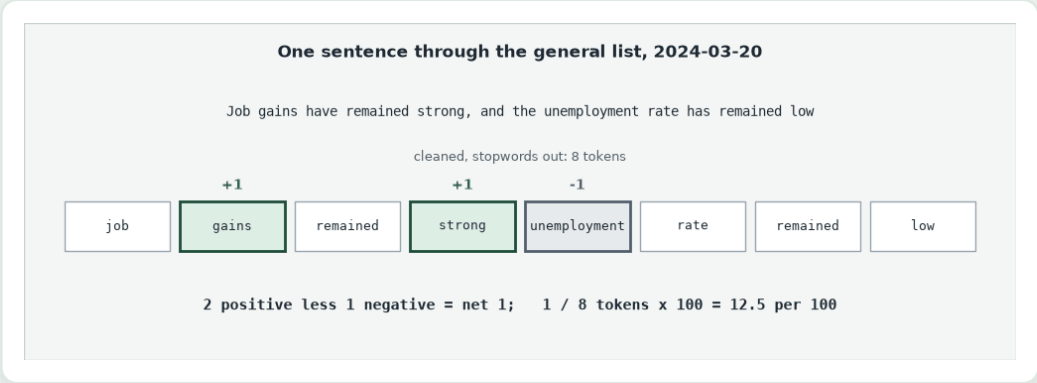
18.2 A score from two word lists

```
positive_words = ['good', 'strong', 'better', 'gains', 'support', 'progress', 'improved',
                  'improvement', 'confidence', 'growth', 'solid', 'stable',
                  'recovery', 'benefit']
negative_words = ['bad', 'weak', 'worse', 'risks', 'inflation', 'unemployment', 'pressures',
                  'restrictive', 'uncertain', 'hardship', 'decline', 'loss', 'debt', 'lower']
first_tokens = statements['kept'].iloc[0]
positive_hits = [token for token in first_tokens if token in positive_words]
negative_hits = [token for token in first_tokens if token in negative_words]
print(statements['date'].iloc[0], len(first_tokens), positive_hits)
print(negative_hits)
```

The statement of 2021-01-27 has 5 positive hits and 14 negative, 10 of them "inflation".

```
positive_hits - negative_hits
TypeError: unsupported operand type(s) for -: 'list' and 'list'
```

Read the last line, as in week 1's Day 3: one list minus another means nothing. The score subtracts the two "len()" counts and divides by the tokens, as the picture does for one sentence.



6 and 17 pieces where the tokens gave 5 and 14: "support" sits inside "supporting", "weak" inside "weakness" and "weaker". A score says which it counts: here, whole tokens.

Excel side by side. Text in C2:C47, the positive words in F2:F15; every version.

Job	Python	Excel
Statements holding a word	`.str.contains('restrictive')`	=COUNTIF(C2:C47, "*restrictive*") returns 4
Pieces of a whole list	`the count() line above`	=SUMPRODUCT((LEN(C2)-LEN(SUBSTITUTE(LOWER(C2),$F$2:$F$15,"")))/LEN($F$2:$F$15)) returns 6

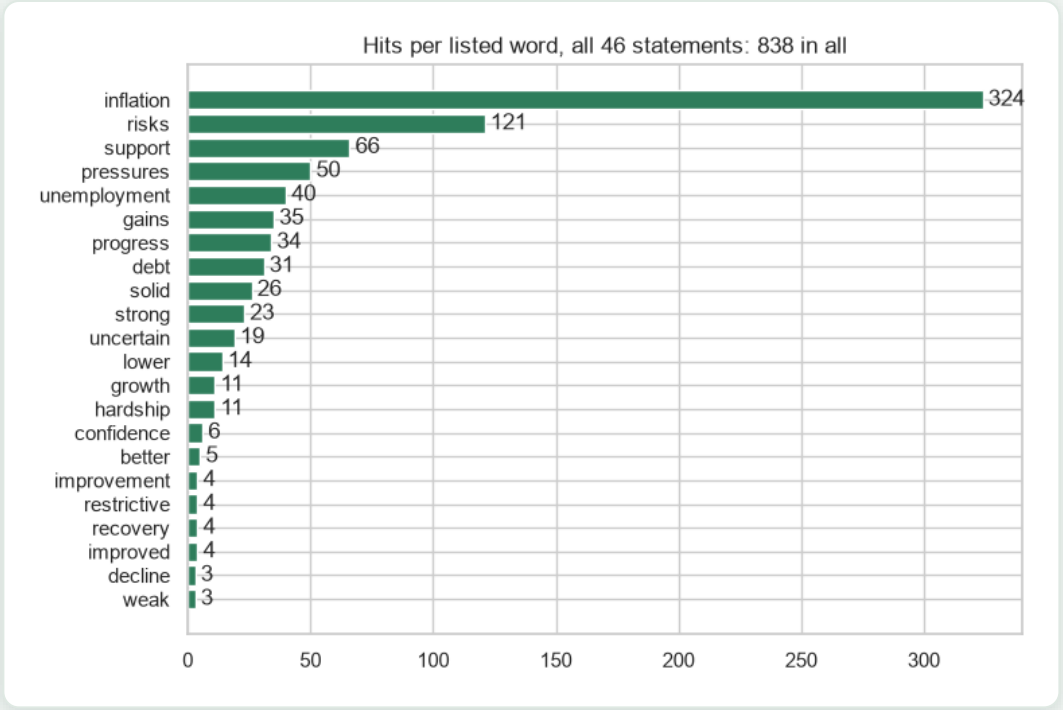
18.3 Which words make the score

```
rows = []
for date, text in zip(statements['date'], statements['text']):
    for piece in text.replace('\n', ' ').split('. '):
        rows.append({'date': date, 'sentence': piece})
sentences = pd.DataFrame(rows)
sentences['kept'] = sentences['sentence'].apply(clean).apply(remove_stop_words)
print(len(sentences), (sentences['date'] == '2024-03-20').sum())
print(sentences[sentences['date'] == '2024-03-20']['sentence'].tail(3).tolist())
1198 27
['Kugler; Loretta J', 'Mester; and Christopher J', 'Waller.']
```

The 1,198 pieces include fragments of names, cut at each middle initial's full stop: a known limit, left in, since a name holds no listed word.

```
def sentences_with(word):
    return sentences[sentences['kept'].apply(lambda tokens: word in tokens)]
def show(table, limit):
    for date, sentence in zip(table['date'][:limit], table['sentence'][:limit]):
        print(date, '|', sentence)
repeated = sentences_with('inflation')['sentence'].value_counts()
print(repeated.iloc[0], '|', repeated.index[0])
```

That sentence is in 43 of 46 statements and is a negative hit every time.



"inflation" makes 324 of the 838 hits, more than every positive word together; "good" and "bad" never hit, so they have no bar.

Terms of the trade, and a sentence with not

```
domain_terms = ['inflation', 'risks', 'restrictive', 'unemployment', 'pressures',
                'strong', 'support']
for word in domain_terms:
    show(sentences_with(word), 1)
seven = sum([hits[word] for word in domain_terms])
print(seven, 'of', sum(hits.values()))
628 of 838
```

Seven words that name things here make 628 of the 838 hits. This is the domain vocabulary problem: a general list carries a word's everyday sense.

Loughran and McDonald, 2011

In 2011 Tim Loughran and Bill McDonald counted negative words in 10-K filings with the Harvard Dictionary, a general list: "Almost three-fourths (73.8%) of the negative word counts according to the Harvard list are attributable to words that are typically not negative in a financial context", such as tax, cost and liability. They built finance word lists instead.

Source: Journal of Finance 66(1), 35-65, https://doi.org/10.1111/j.1540-6261.2010.01625.x. Their dictionary (https://sraf.nd.edu/loughranmcdonald-master-dictionary/) needs a licence for commercial use, so it stays out of this course.

```
all_not = sentences[sentences['sentence'].str.contains(' not ')].drop_duplicates('sentence')
show(all_not, len(all_not))
```

date	sentence
2021-07-28	The sectors most adversely affected by the pandemic have shown improvement but have not fully recovered
2024-01-31	The Committee does not expect it will be appropriate to reduce the target range until it has gained greater confidence that inflation is moving sustainably toward 2 percent
2025-07-30	Absent and not voting was Adriana D
2026-04-29	Logan, who supported maintaining the target range for the federal funds rate but did not support inclusion of an easing bias in the statement at this time.

18.4 A published lexicon: VADER

Where It Comes From	
Cite as	Hutto and Gilbert (2014), VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text, nltk's citation (https://www.nltk.org/api/nltk.sentiment.vader.html)
Built for	"a lexicon and rule-based sentiment analysis tool that is specifically attuned to sentiments expressed in social media" (https://github.com/cjhutto/vaderSentiment)
Ratings, licence	human raters, -4 to 4; MIT License (same page)

```
analyzer = SentimentIntensityAnalyzer()
print(len(analyzer.lexicon))
look_up = ['good', 'bad', 'strong', 'weak', 'inflation', 'securities', 'treasury']
for word in look_up:
    print(word, analyzer.lexicon.get(word, 'absent'))
7502
good 1.9
bad -2.5
strong 2.3
weak -1.9
inflation absent
securities 1.2
treasury 0.8
```

```
sentence = sentences[sentences['date'] == '2024-03-20']['sentence'].iloc[1]
print(sentence)
print(analyzer.polarity_scores(sentence))
Job gains have remained strong, and the unemployment rate has remained low
{'neg': 0.267, 'neu': 0.428, 'pos': 0.305, 'compound': 0.1779}
```

The compound, 0.1779, is "the VADER score" from here on.

Of 7,502 rated words, "inflation" is not one: the word behind 324 hits counts for nothing. Yet "securities" and "treasury" are rated, names of instruments read as everyday words: the domain problem has moved.

Excel side by side. The lexicon on a sheet "lexicon" (words in A, ratings in B), VADER's scores in E2:E47.

Job	Python	Excel
Look up a rating	`.lexicon.get('strong', 0)`	=XLOOKUP("strong", lexicon!A:A, lexicon!B:B, 0) returns 2.3, and 0 for a missing word
The label	`label_score(compound)`	=IF(E2>=$H$1,"positive",IF(E2<=-$H$1,"negative","neutral")), the cut-off in H1
Labels at a cut-off	`value_counts()`	=COUNTIF($E$2:$E$47,">="&H2) and "<="&-H2 return 29 and 8 at 0.5

"XLOOKUP" is in Microsoft 365 and Excel 2021 on.

The words VADER read, and a label at a cut-off

```
from nltk.sentiment.vader import SentiText
def rated_words(text):
    split_text = SentiText(text, analyzer.constants.PUNC_LIST,
                           analyzer.constants.REGEX_REMOVE_PUNCTUATION)
    tokens = [token.lower() for token in split_text.words_and_emoticons]
    return [(token, analyzer.lexicon[token]) for token in tokens if token in analyzer.lexicon]
print(vader_score(all_not['sentence'].iloc[1]), rated_words(all_not['sentence'].iloc[1]))
print(analyzer.lexicon.get('backed'), rated_words('agency mortgage-backed securities'))
0.8126 [('gained', 1.6), ('greater', 1.5), ('confidence', 2.3)]
0.1 [('securities', 1.2)]
```

"backed" is rated, yet VADER never scores it inside "mortgage-backed", while our "clean()" would split it off.

The authors' cut-offs: 0.05 or more is positive, -0.05 or less negative, anything between neutral.

```
def label_score(compound, cutoff=0.05):
    if compound >= cutoff:
        return 'positive'
    elif compound <= -cutoff:
        return 'negative'
    return 'neutral'
statements['vader_label'] = statements['vader'].apply(label_score)
label_order = ['negative', 'neutral', 'positive']
print(statements['vader_label'].value_counts().reindex(label_order, fill_value=0))
vader_label
negative    11
neutral      1
positive    34
Name: count, dtype: int64
```

vader	negative	neutral	positive
0.05	11	1	34
0.25	10	3	33
0.5	8	9	29
0.75	6	17	23
0.9	1	30	15

Neutral climbs from 1 to 30 and positive falls from 34 to 15; the scores never changed.

The cut-off moves the counts as much as the text does. A list built for this text is next.

18.5 A word list built for this text

Two groups, defined by what the words say: firm (activity rising or solid) and soft (falling or weak).

```
all_counts = Counter()
for tokens in statements['kept']:
    all_counts.update(tokens)
print(len(all_counts), [all_counts[word] for word in ['booming', 'contracted', 'slumped']])
firm_words = ['expanding', 'expand', 'expanded', 'solid', 'strong', 'robust', 'strengthened',
              'picked', 'resilient', 'gains']
soft_words = ['slowed', 'moderated', 'declined', 'weak', 'weakness', 'weaker', 'softened']
print(sum([all_counts[word] for word in firm_words]))
print(sum([all_counts[word] for word in soft_words]))
507 [0, 0, 0]
133
26
```

```
def domain_score(tokens):
    return list_score(tokens, firm_words, soft_words)
statements['domain'] = statements['kept'].apply(domain_score)
print(round(statements['domain'].mean(), 2), (statements['domain'] < 0).sum())
print(statements['domain'].round(2).tolist()[-4:], statements['n_kept'].tolist()[-4:])
1.3 3
```

Only 3 statements fall below zero.

Of 507 different tokens, the firm words hit 133 times and the soft 26; "booming", "contracted" and "slumped" never appear. It is one teaching list among many, printed in full.

```
show(sentences_with('declined').drop_duplicates('sentence'), 5)
```

In none of the four does "declined" describe activity: three are about the unemployment rate, one about reserve balances.

Excel side by side

Firm words in J2:J11, soft in K2:K8: 18.2's "SUMPRODUCT" on each, firm less soft. Pieces bite harder here: "expanding" counts twice, as itself and as "expand", and "against" holds "gains".

Built from the file, the list is still blind to which word describes which.

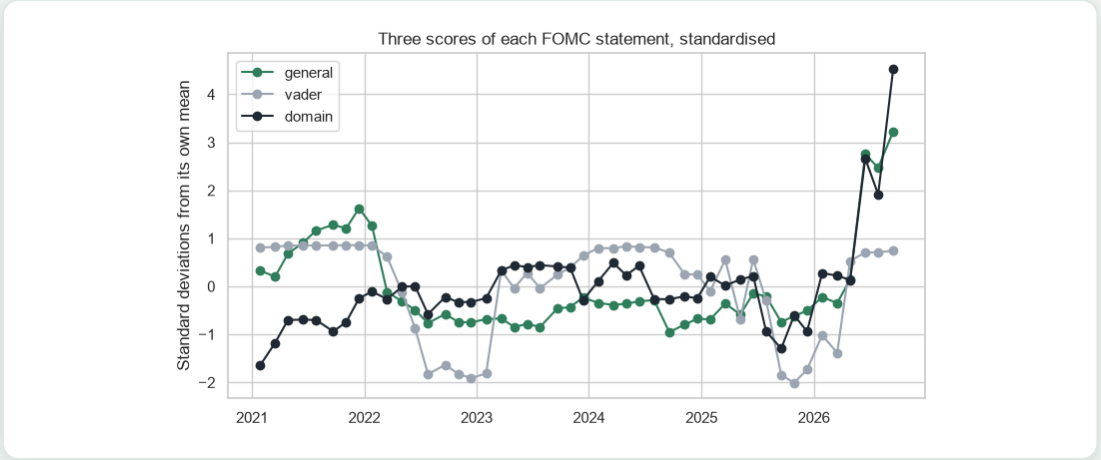
18.6 Three scores side by side

```
scores = statements.set_index('meeting')[['general', 'vader', 'domain']]
scores.corr().style.format('{:.2f}').background_gradient(cmap='Greens', vmin=-1, vmax=1)
```

	general	vader	domain
general	1.00	0.48	0.51
vader	0.48	1.00	0.21
domain	0.51	0.21	1.00

The pairs correlate at 0.48, 0.51 and 0.21: three lists read the same texts and agree loosely.

```
changes = scores.diff()
print(changes.corr().round(2))
same_way = (changes['general'] > 0) == (changes['domain'] > 0)
print(same_way.iloc[1:].sum(), 'of', len(changes) - 1)
21 of 45
```



The lines do not track: the first statement sits above its mean on two lists and 1.65 below on the third.

The changes correlate at 0.33, 0.62 and 0.20, yet general and domain moved the same way only 21 times in 45.

Excel side by side. The general scores in D2:D47, VADER's in E2:E47; every version.

Job	Python	Excel
z-score	`(scores - scores.mean()) / scores.std()`	=STANDARDIZE(D2, AVERAGE($D$2:$D$47), STDEV.S($D$2:$D$47)) returns 0.33
Correlation	`scores.corr()`	=CORREL(D2:D47, E2:E47) returns 0.48
Change from the statement before	`scores.diff()`	=D3-D2 in row 3, filled down; row 2 has none

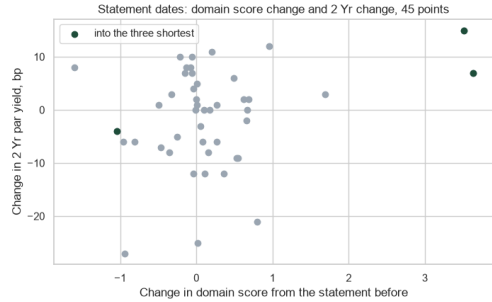
18.7 Scores and yields on statement dates

```
yields = pd.read_csv(data_folder + 'treasury_par_yields.csv',
                     parse_dates=['date'], index_col='date')
yield_changes = (yields[['2 Yr', '10 Yr']].diff() * 100).round(0)
joined = changes.join(yield_changes, how='left')
print(len(joined), len(joined.dropna()))
46 45
```

45 of 46 rows are complete: only the first statement lacks a change, so every date found its yields.

```
without_last = joined.iloc[:-3]
print(without_last.corr().loc[['general', 'vader', 'domain'], ['2 Yr', '10 Yr']].round(2))
         2 Yr  10 Yr
general -0.07  -0.11
vader   -0.07  -0.04
domain   0.07   0.10
```

On 42 points all six fall between -0.11 and 0.10: three points carried the 0.27.



	2 Yr	10 Yr
general	0.19	0.07
vader	-0.06	-0.03
domain	0.27	0.16

All six fall between -0.06 and 0.27 on 45 points. Near zero is a result, and it is reported as one.

Excel side by side. The yields on a sheet "yields", dates in A, the 2 Yr in I.

Job	Python	Excel
Change in bp	`.diff() * 100`	=(I3-I2)*100 in P3, filled down
Each statement date's change	`.join(yield_changes, how='left')`	=XLOOKUP(A2, yields!$A$2:$A$2941, yields!$P$2:$P$2941); a missing date gives #N/A, Python's NaN

Predict first: decide, then run each in the notebook

Predict 1

The statement of 2026-09-16 has a net of 4 hits on 79 tokens; 2021-01-27 has -9 on 304. Which is further from zero, per 100 tokens? A) 2021-01-27, since 9 hits are more than 4 B) 2026-09-16 C) the same distance

```
def list_score(tokens, plus_words, minus_words):
    plus = len([token for token in tokens if token in plus_words])
    minus = len([token for token in tokens if token in minus_words])
    return (plus - minus) / len(tokens) * 100
def general_score(tokens):
    return list_score(tokens, positive_words, negative_words)
last_tokens = statements['kept'].iloc[-1]
print(round(general_score(last_tokens), 2), round(general_score(first_tokens), 2))
print(round(100 / len(last_tokens), 2), round(100 / len(first_tokens), 2))
```

Predict 2

Take the seven off both lists and score again. With them, 42 of 46 statements sit below zero. How many now? Write down a number.

```
positive_rest = [word for word in positive_words if word not in domain_terms]
negative_rest = [word for word in negative_words if word not in domain_terms]
general_rest = statements['kept'].apply(
    lambda tokens: list_score(tokens, positive_rest, negative_rest))
print((general_rest < 0).sum(), round(general_rest.mean(), 2))
```

Predict 3

Cleaned and matched against the two lists, how many hits does the 2024-01-31 sentence give? A) 2: "confidence" and "inflation" B) 1: "not" cancels "confidence" C) 0: the sentence describes no change

```
tokens = all_not['kept'].iloc[1]
print(tokens)
print('not' in stop_words)
print([token for token in tokens if token in positive_words + negative_words])
```

Predict first: VADER and the domain list

Predict 4

VADER gave one sentence 0.1779. Score the whole first statement. Write down a number between -1 and 1.

```
def vader_score(text):
    return analyzer.polarity_scores(text)['compound']
print(vader_score(statements['text'].iloc[0]))
```

Predict 5

The 2021-07-28 sentence printed in 18.3 scores 0.0037 as published. Score its cleaned tokens, joined by spaces. The new score is: A) the same: the rated words did not change B) higher C) lower

```
raw = all_not['sentence'].iloc[0]
cleaned = ' '.join(all_not['kept'].iloc[0])
print(vader_score(raw), vader_score(cleaned))
print('not' in stop_words, 'but' in stop_words)
```

Predict 6

One sentence of 2024-09-18 holds "gains" and "slowed" and no other domain word. What does it add to the statement's net hits? A) -1 B) 0 C) +1

```
both = sentences_with('gains')
both = both[both['kept'].apply(lambda tokens: 'slowed' in tokens)]
one = both[both['date'] == '2024-09-18']
show(one, 1)
print([token for token in one['kept'].iloc[0] if token in firm_words + soft_words])
```

The note, answered

The note asked	Answer
One number, three ways	42, 12 and 3 of 46 below zero
The words behind them	inflation is 324 of 838 general hits
Do they agree	loosely: 0.48, 0.51, 0.21
Against yields	near zero, on 45 points and on 42

Caution	Why
Few points	three left out moved the figure
Other news	nothing here separates the statement from the rest of the day
A daily yield	quotes "at or near 3:30 PM each trading day"
	(https://home.treasury.gov/policy-issues/financing-the-government/interest-rate-statistics/treasury-yield-curve-methodology), not the minutes after a statement
This file only	a description of 45 pairs, not a cause

Next, From Notebook to Module moves week 3's cleaning routine into a file of functions.